

Introduction to the tidyverse in R

DSC 152: Week 2 Discussion

Zida Jin

University of California, San Diego

2026.10.05

Examples use the public palmerpenguins dataset.

Today's goals

By the end of this session you should be able to:

- 1 Explain what *tidy data* means
- 2 Read and write a pipeline with `|>`
- 3 Use core `dplyr` verbs
- 4 Reshape data between wide and long formats
- 5 Turn a table into a basic `ggplot2` plot

Before we start

```
install.packages(c("tidyverse", "palmerpenguins"))
```

Roadmap

- 1 Why the tidyverse?
- 2 Tibbles and importing data
- 3 The pipe
- 4 dplyr: core verbs
- 5 tidyr: reshaping
- 6 ggplot2 demo
- 7 Wrap-up

What is the tidyverse?

A collection of R packages that share one design philosophy and grammar.

Core packages (loaded by
`library(tidyverse)`)

- `readr`: import
- `tibble`: data frames
- `dplyr`, `tidyr`: wrangle
- `ggplot2`: visualise
- `purrr`: iterate
- `stringr`, `forcats`, `lubridate`: strings, factors, dates

Why use it?

- Consistent function names and arguments
- Code reads like a sentence
- Data frame in, data frame out
- Large community and documentation

Tidy data

Three rules:

- 1 Each **variable** is a column
- 2 Each **observation** is a row
- 3 Each **value** is a cell

Tidy data lets every tidyverse function assume the same shape.

A tidy example

name	quiz	score
Alice	1	10
Alice	1	8
Bob	2	9
Bob	2	8

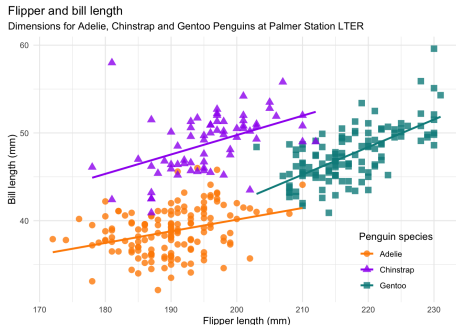
An untidy example

name	quiz1_score	quiz2_score
Alice	10	8
Bob	9	8

Getting started

```
library(tidyverse)
library(palmerpenguins) # sample dataset used for discussion
```

palmerpenguins is a built-in dataset containing the bill and flipper lengths for three species.



Roadmap

- 1 Why the tidyverse?
- 2 Tibbles and importing data**
- 3 The pipe
- 4 dplyr: core verbs
- 5 tidyr: reshaping
- 6 ggplot2 demo
- 7 Wrap-up

Importing with readr

```
# CSV: Comma-Separated Values
cars <- read_csv(readr_example("mtcars.csv"))
#> Rows: 32 Columns: 11
#> Column specification: dbl (11): mpg, cyl, disp, ...
```

read_csv() (readr)

- Faster on large files
- Reports the column types it guessed
- Returns a tibble

read.csv() (base R)

- Silent type guessing
- Returns a data.frame

Tibble: a modern data frame

```
glimpse(penguins)
#> Rows: 344
#> Columns: 8
#> $ species      <fct> Adelie, Adelie, Adelie, ...
#> $ island       <fct> Torgersen, Torgersen, ...
#> $ bill_length_mm <dbl> 39.1, 39.5, 40.3, NA, ...
#> ...
```

- Prints only the first 10 rows and the columns that fit
- Shows each column's type: <fct>, <dbl>, <int>, <chr>
- Stricter than data.frame: no partial name matching, clearer warnings

Roadmap

- 1 Why the tidyverse?
- 2 Tibbles and importing data
- 3 The pipe**
- 4 dplyr: core verbs
- 5 tidyr: reshaping
- 6 ggplot2 demo
- 7 Wrap-up

The pipe operator: `|>`

Nested (read inside out)

```
arrange(filter(penguins, species == "Gentoo"), body_mass_g)
```

Piped (read top to bottom)

```
penguins |>  
  filter(species == "Gentoo") |>  
  arrange(body_mass_g)
```

`x |> f(y)` is equivalent to `f(x, y)`.

|> or %>%?

Native pipe |>

- Built into R (version 4.1.0 or later)
- No package needed
- Used in *R for Data Science*, 2nd edition

magrittr pipe %>%

- Older, common in existing code and online answers
- A few extra features

Rule for discussion sections

Pick one and use it consistently. We use |>.

Roadmap

- 1 Why the tidyverse?
- 2 Tibbles and importing data
- 3 The pipe
- 4 dplyr: core verbs**
- 5 tidyr: reshaping
- 6 ggplot2 demo
- 7 Wrap-up

The dplyr grammar

Acts on	Verb	What it does
Rows	<code>filter()</code>	keep rows that meet a condition
	<code>arrange()</code>	reorder rows
Columns	<code>select()</code>	keep or drop columns
	<code>mutate()</code>	create or modify columns
Groups	<code>summarise()</code>	collapse to summary values
	<code>count()</code>	count rows per group

Every verb takes a data frame first and returns a data frame, so they chain together with `|>`.

Rows: filter() and arrange()

```
penguins |>  
  filter(species == "Adelie", body_mass_g > 4000) |>  
  arrange(desc(body_mass_g))
```

- Commas inside filter() mean **and**; use | for **or**
- Use == to test equality (NOT =)
- %in% tests membership: filter(island %in% c("Biscoe", "Dream"))
- desc() sorts from largest to smallest

Columns: `select()` and `mutate()`

```
penguins |>  
  select(species, island, body_mass_g) |>  
  mutate(body_mass_kg = body_mass_g / 1000)
```

- Helpers inside `select()`: `starts_with()`, `ends_with()`, `where()`
- Drop a column with a minus sign: `select(-year)`
- `mutate()` can use columns created earlier in the same call

Groups: summarise() and count()

```
penguins |>
  summarise(
    mean_mass = mean(body_mass_g, na.rm = TRUE),
    n         = n(),
    .by       = species
  )

penguins |> count(species, island)
```

- `.by` = groups for this one call only
- `na.rm = TRUE` ignores missing values

Exercise 1

Task

For penguins with a recorded sex, find the mean flipper length for each species and sex, sorted from longest to shortest.

Structure Hint:

```
[DATASET] |>
  [WITH A RECORDED SEX] |>
  [GROUPS FUNCTION](
    mean_flipper = [AVERAGE FUNCTION](flipper_length_mm, na.rm = TRUE),
    [FOR EACH SPECIES & SEX]
  ) |>
  [SORT IN DECESENDING ORDER]
```

Exercise 1: solution

Task

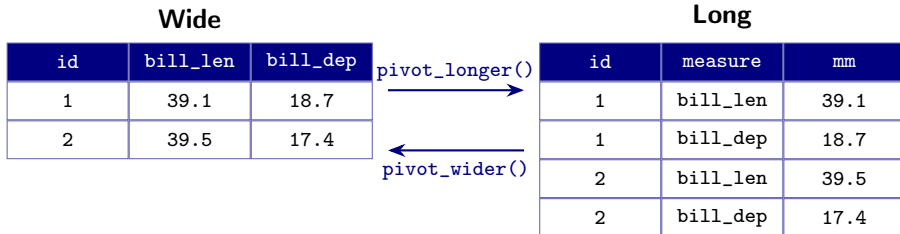
For penguins with a recorded sex, find the mean flipper length for each species and sex, sorted from longest to shortest.

```
penguins |>
  filter(!is.na(sex)) |>
  summarise(
    mean_flipper = mean(flipper_length_mm, na.rm = TRUE),
    .by = c(species, sex)
  ) |>
  arrange(desc(mean_flipper))
```

Roadmap

- 1 Why the tidyverse?
- 2 Tibbles and importing data
- 3 The pipe
- 4 dplyr: core verbs
- 5 tidyr: reshaping**
- 6 ggplot2 demo
- 7 Wrap-up

Wide versus long



Long format is usually what ggplot2 and grouped summaries need.

pivot_longer() and pivot_wider()

```
bills_long <- penguins |>
  mutate(id = row_number()) |>
  select(id, species, bill_length_mm, bill_depth_mm) |>
  pivot_longer(
    cols      = c(bill_length_mm, bill_depth_mm),
    names_to  = "measure",
    values_to = "mm"
  )

bills_long |>
  pivot_wider(names_from = measure, values_from = mm)
```

drop_na() removes rows containing missing values.

Roadmap

- 1 Why the tidyverse?
- 2 Tibbles and importing data
- 3 The pipe
- 4 dplyr: core verbs
- 5 tidyr: reshaping
- 6 ggplot2 demo**
- 7 Wrap-up

The grammar of graphics

Data + aesthetic mappings (aes()) + geometric objects (geom_*())

```
ggplot(penguins,  
       aes(x = flipper_length_mm, y = body_mass_g,  
           colour = species)) +  
  geom_point() +  
  labs(x = "Flipper length (mm)", y = "Body mass (g)")
```

Warning

Layers are joined with +, not |>.

From pipeline to plot

Plot a bar plot of the mean mass (g) by species.

```
penguins |>
  summarise(mean_mass = mean(body_mass_g, na.rm = TRUE),
            .by = species) |>
  ggplot(aes(x = species, y = mean_mass)) +
  geom_col(fill = "navy") +
  labs(x = NULL, y = "Mean body mass (g)")
```

Roadmap

- 1 Why the tidyverse?
- 2 Tibbles and importing data
- 3 The pipe
- 4 dplyr: core verbs
- 5 tidyr: reshaping
- 6 ggplot2 demo
- 7 Wrap-up**

Exercise 2

Build one pipeline that:

- ① keeps only female penguins
- ② computes mean flipper length per island
- ③ sorts islands from longest to shortest
- ④ plots the result as a bar chart

Common pitfalls

- `=` instead of `==` inside `filter()`
- Forgetting `na.rm = TRUE`, so a summary returns NA
- Forgetting `ungroup()` after `group_by()` (if used)
- Using `|>` between `ggplot2` layers instead of `+`
- Forgetting to save the result: `x <- x |> ...`

Today

- Tidy data: variables, observations, values
- `|>` chains steps
- dplyr verbs for rows, columns, groups
- tidyr for reshaping
- ggplot2 for plotting

Further reading

- *R for Data Science* (2nd ed.):
<https://r4ds.hadley.nz>
- Package overview:
<https://www.tidyverse.org/packages>
- Posit cheatsheets: <https://posit.co/resources/cheatsheets>

Backup: exercise solution

```
penguins |>
  filter(sex == "female") |>
  summarise(mean_flipper = mean(flipper_length_mm, na.rm = TRUE),
            .by = island) |>
  arrange(desc(mean_flipper)) |>
  ggplot(aes(x = fct_reorder(island, mean_flipper, .desc = TRUE),
            y = mean_flipper)) +
  geom_col(fill = "navy") +
  labs(x = "Island", y = "Mean flipper length (mm)")
```

`fct_reorder()` (forcats) keeps the bar order; `ggplot2` ignores row order otherwise.